

Последователността на работа в тримерен режим може да бъде сведена до:

- I. Инициализация на 3D**
- II. Дефиниране на тип проекция**
- III. Задаване на фонов цвят за екрана**
- IV. Задаване на подсветка (видимо осветление)**
- V. Дефиниране на източник на осветление (динамична светлина)**
- VI. Режим на отклик на материала**
- VII. Определяне на позиция на наблюдение на сцената**
- VIII. Избор на метод на свързване на върховете на фигурата**
- IX. Начало на фигура**
- X. Задаване на върхове и нормали**
- XI. Задаване на цвят на фигурата**
- XII. Край на фигурата**
- XIII. Ориентация на фигурата**
- XIV. Визуализация**

Средства за реализация

В използваната библиотека OpenGL тримерната сцена съществува резидентно в паметта и може да бъде наблюдавана под различен ъгъл, в различно ограничително пространство. Това предполага специално форматиране на буферното пространство, използвано за изчертаване, размер на пространството от сцената и посока от която гледа наблюдателят. Всеки обект в сцената се моделира чрез върхове оформящи каркасен модел, запълване на многоъгълници и нормали, чрез които се осигурява осветяване по метода на Гуро. Наличните сплайни и равнини дават възможност за по-ефективно моделиране на сложни обекти чрез задаване на точки от контурите на повърхнината и управление на кривината с помощта на набор от възли. За да има видимост на обектите, е достатъчно да има подсветка на сцената, която да осигури визуализация без сенки, ръбове и тримерни детайли. Библиотеката дава възможност за осветяване с различни светлинни източници, осигуряващи обемност на видимостта и динамично изменение на осветеността според способностите за отражение на обекта и материала, от който са направени.

OpenGL – пакет библиотеки

(<https://msdn.microsoft.com/en-us/library/windows/desktop/dd374293%28v=vs.85%29.aspx>)

Съдържание на използвания пакет:

GL – ядро на библиотеката. Моделира обект чрез задаване на геометрични примитиви като точки, линии и полигони.

GLU – OpenGL Utility Library : надгражда ядрото на OpenGL за създаване на по-сложни модели.

GLUT – OpenGL Utilities Toolkit: поддържа взаимодействието между операционната система (като създаване на прозорец, манипулатори и обслужване на мишката) и по-сложни модели (като сфера и торус).

Всеки софтуерен пакет се състои от:

1. Заглавен файл: "gl.h" – за ядрото на OpenGL, "glu.h" за GLU и "glut.h" за GLUT, обикновено съхранени в "include/GL" директория.
2. Статични библиотеки: например за Win32, "libopengl32.a" - за ядрото на OpenGL, "libglu32.a" – за GLU, "libglut32.a" (или "glut32.lib") за GLUT, съхранявани в "lib" директория.
3. Опционални динамични библиотеки: например "glut32.dll" за GLUT под Win32, съхранявани в "bin" или "c:\windows\system32".

Описание на библиотеките:

В библиотеките се ползват 13 типа променливи за:

- Трансформации
- Оцветяване
- Осветяване
- Растеризация
- Текстуриране
- Контрол на кадровите буфери
- Пиксел
- Изчислител
- Подсказка
- Зависими от приложението
- Зависима от приложението дълбочина на пиксела
- Разнородни

Функциите на OpenGL започват с **gl**, завършват с число и буква (или само буква). Буквата указва типа на приеманите аргумент: f - float, d - double, i - integer.

Ако присъства число, то указва броя на аргументите, извиквани от функцията. Ако присъства буквата „v” накрая, това означава, че функцията приема аргумент – указател към масив от съответния тип, който съдържа съответния брой стойности. Ако в скоби {} са посочени няколко букви или няколко числа, то функцията има няколко разновидности: може да се напише с всяко едно от числата или буквите в скобите или масив, ако има [v] .

По-долу е даден списък на функциите от библиотеките.

GL функции

(<https://msdn.microsoft.com/en-us/library/dd374211%28v=vs.85%29.aspx>)

glAccum	glClearDepth
glAddSwapHintRectWIN	glClearIndex
glAlphaFunc	glClearStencil
glAreTexturesResident	glClipPlane
glArrayElement	glColor
glBegin, glEnd	glColorMask
glBindTexture	glColorMaterial
glBitmap	glColorPointer
glBlendFunc	glColorTableEXT
glCallList	glColorSubTableEXT
glCallLists	glCopyPixels
glClear	glCopyTexImage1D
glClearAccum	glCopyTexImage2D
glClearColor	glCopyTexSubImage1D

glCopyTexSubImage2D

glCullFace

glDeleteLists

glDeleteTextures

glDepthFunc

glDepthMask

glDepthRange

glDisable

glDisableClientState

glDrawArrays

glDrawBuffer

glDrawElements

glDrawPixels

glEdgeFlag

glEdgeFlagPointer

glEnable

glEnableClientState

glEvalCoord

glEvalMesh1, glEvalMesh2

glEvalPoint1, glEvalPoint2

glFeedbackBuffer

glFinish

glFlush

glFogf, glFogi, glFogfv, glFogiv

glFrontFace

glFrustum

glGenLists

glGenTextures

**glGetBooleantv, glGetDoublev,
glGetFloatv, glGetIntegerv**

glGetClipPlane

glGetColorTableEXT

**glGetColorTableParameterfvEXT
,
glGetColorTableParameterivEXT**

glGetError

glGetLightfv, glGetLightiv

**glGetMapdv, glGetMapfv,
glGetMapiv**

glGetMaterialfv, glGetMaterialiv

**glGetPixelMapfv,
glGetPixelMapuiv,
glGetPixelMapusv**

glGetPointerv

glGetPolygonStipple

glGetString

glGetTexEnvfv

**glGetTexGendv, glGetTexGenfv,
glGetTexGeniv**

glGetTexImage

**glGetTexLevelParameterfv,
glGetTexLevelParameteriv**

**glGetTexParameterfv,
glGetTexParameteriv**

glHint

glIndex

glIndexMask

glIndexPointer

glInitNames

glInterleavedArrays

glIsEnabled

glIsList

glIsTexture

**glLightf, glLighti, glLightfv,
glLightiv**

**glLightModelf, glLightModeli,
glLightModelfv, glLightModeliv**

glLineStipple

glLineWidth

glListBase

glLoadIdentity

glLoadMatrixd, glLoadMatrixf

glLoadName

glLogicOp

glMap1d, glMap1f

glMap2d, glMap2f

**glMapGrid1d, glMapGrid1f,
glMapGrid2d, glMapGrid2f**

**glMaterialf, glMateriali,
glMaterialfv, glMaterialiv**

glMatrixMode

glMultMatrixd, glMultMatrixf

glNewList, glEndList

glNormal

glNormalPointer

glOrtho

glPassThrough

**glPixelMapfv, glPixelMapuiv,
glPixelMapusv**

glPixelStoref, glPixelStorei

glPixelTransferf, glPixelTransferi

glPixelZoom

glPointSize	glShadeModel
glPolygonMode	glStencilFunc
glPolygonOffset	glStencilMask
glPolygonStipple	glStencilOp
glPrioritizeTextures	glTexCoord
glPushAttrib, glPopAttrib	glTexCoordPointer
glPushClientAttrib, glPopClientAttrib	glTexEnvf, glTexEnvi, glTexEnvfv, glTexEnviv
glPushMatrix, glPopMatrix	glTexGend, glTexGenf, glTexGeni, glTexGendv, glTexGenfv, glTexGeniv
glPushName, glPopName	
glRasterPos	glTexImage1D
glReadBuffer	glTexImage2D
glReadPixels	glTexParameterf, glTexParameteri, glTexParameterfv, glTexParameteriv
glRectd, glRectf, glRecti, glRects, glRectdv, glRectfv, glRectiv, glRectsv	glTexSubImage1D
glRenderMode	glTexSubImage2D
glRotated, glRotatef	glTranslated, glTranslatef
glScaled, glScalef	glVertex
glScissor	glVertexPointer
glSelectBuffer	glViewport

GLU функции

(<https://msdn.microsoft.com/en-us/library/dd374158%28v=vs.85%29.aspx>)

gluBeginCurve, gluEndCurve
gluBeginPolygon, gluEndPolygon
gluBeginSurface, gluEndSurface
gluBeginTrim, gluEndTrim
gluBuild1DMipmaps
gluBuild2DMipmaps
gluCylinder
gluDeleteNurbsRenderer
gluDeleteQuadric
gluDeleteTess
gluDisk
gluErrorString
gluGetNurbsProperty
gluGetString
gluGetTessProperty
gluLoadSamplingMatrices
gluLookAt
gluNewNurbsRenderer
gluNewQuadric
gluNewTess
gluNextContour
gluNurbsCallback
gluNurbsCurve
gluNurbsProperty

gluNurbsSurface
gluOrtho2D
gluPartialDisk
gluPerspective
gluPickMatrix
gluProject
gluPwlCurve
gluQuadricCallback
gluQuadricDrawStyle
gluQuadricNormals
gluQuadricOrientation
gluQuadricTexture
gluScaleImage
gluSphere
gluTessBeginContour,
gluTessEndContour
gluTessBeginPolygon,
gluTessEndPolygon
gluTessCallback
gluTessNormal
gluTessProperty
gluTessVertex
gluUnProject

GLUT функции

(<https://www.opengl.org/resources/libraries/glut/spec3/spec3.html>)

glutInit
glutInitWindowSize
glutInitWindowPosition
glutInitDisplayMode
glutMainLoop
glutCreateWindow
glutCreateSubWindow
glutSetWindow
glutDestroyWindow
glutPostRedisplay
glutSwapBuffers

glutPositionWindow
glutReshapeWindow
glutFullScreen
glutPopWindow
glutPushWindow
glutShowWindow
glutHideWindow
glutIconifyWindow
glutSetWindowTitle,
glutSetIconTitle
glutSetCursor

glutEstablishOverlay
glutUseLayer
glutRemoveOverlay
glutPostOverlayRedisplay
glutShowOverlay,
glutHideOverlay
glutCreateMenu
glutSetMenu,
glutGetMenu
glutDestroyMenu
glutAddMenuEntry
glutAddSubMenu
glutChangeToMenuEntry
glutChangeToSubMenu
glutRemoveMenuItem
glutAttachMenu,
glutDetachMenu
glutDisplayFunc
glutOverlayDisplayFunc
glutReshapeFunc
glutKeyboardFunc
glutMouseFunc
glutMotionFunc,
glutPassiveMotionFunc
glutVisibilityFunc
glutEntryFunc
glutSpecialFunc
glutSpaceballMotionFunc
glutSpaceballRotateFunc
glutSpaceballButtonFunc
glutButtonBoxFunc
glutDialsFunc
glutTabletMotionFunc
glutTabletButtonFunc
glutMenuStatusFunc
glutIdleFunc
glutTimerFunc
glutSetColor
glutGetColor
glutCopyColormap
glutGet
glutLayerGet
glutDeviceGet

glutGetModifiers
glutExtensionSupported
glutBitmapCharacter
glutBitmapWidth
glutStrokeCharacter
glutStrokeWidth
glutSolidSphere, glutWireSphere
glutSolidCube, glutWireCube
glutSolidCone, glutWireCone
glutSolidTorus, glutWireTorus
glutSolidDodecahedron,
glutWireDodecahedron
glutSolidOctahedron,
glutWireOctahedron
glutSolidTetrahedron,
glutWireTetrahedron
glutSolidIcosahedron,
glutWireIcosahedron
glutSolidTeapot, glutWireTeapot

Интегриране на библиотеките към проект

След версия 7 на Windows основните библиотеки от пакета са вградени. GLUT е допълнителна към пакета, затова тя се инсталира към пакета самостоятелно. Състои се от три файла - glut32.dll, glut32.lib и glut.h, като всеки от тях трябва да бъде разположен на правилното място:

- **glut32.dll** – се поставя в папката Windows\system32
- **glut32.lib** – съответно при останалите *.lib файлове в папката LIB на компилатора: C:/Microsoft Visual Studio / VC / LIB , където C:/ е името на съответното дисковото устройство.
- **glut.h** – при останалите хедърни файлове в папката Include. Директория: C:/Microsoft Visual Studio / VC / Include / GL
- **или glut32.dll, glut32.lib и glut.h библиотеки се поставят в директорията на изпълнимия файл**

Директориите на библиотеките се свързват с проекта чрез избор в: Directories - Include files и Library files. За интегриране на библиотеките е достатъчно те да бъдат обявени в началото на основния файл с *include* директива.

Поетапното изпълнение на нужната последователност за създаване и визуализация на обекти се осъществява по указанията по-долу.

Инициализацията на 3D зависи от вида на разработваното приложение. За вариант конзолно приложение създаването и обслужването на графичен екран използва възможностите на GL и GLUT библиотеките. При **Win32 API** проект създаването и обслужването на екрана се осъществява чрез GL библиотеката през системните съобщения. Таблица 3 показва връзката между процесите при обслужване на тримерна графика и анимация, съобщенията и командите към Windows системата.

Таблица 3

Обслужвани процеси в 3D графиката	Обслужващи системни съобщения и команди
Създаване и използване на контекст за рендиране	wglCreateContext, wglDeleteContext, wglMakeCurrent
Заявка и селекция на пикселния формат	ChoosePixelFormat, SetPixelFormat
Връзка със системните съобщения	WM_CREATE, WM_DESTROY, WM_SIZE, WM_PAINT

Инициализацията на 3D графичен режим се постига чрез създаване на контекст за рендиране към екрана при форматирано пикселно пространство с параметър за дълбочина.

Обслужване при системни съобщения **WM_CREATE**, **WM_DESTROY** и **WM_SIZE**

WM_CREATE – използва се за създаване на екран с вграден параметър за дълбочина. В контекста на устройството върху екрана се задава формат на пикселите, според който се рендират фигурите. Управлението се осъществява чрез манипулатори:

HDC – манипулатор към контекст на устройство, чрез който операциите се насочват към избрания екран;

HGLRC – манипулатор към контекст на рендиране, който се създава върху вече форматираното пикселно пространство;

и конструктор на пикселния формат описан с:

PIXELFORMATDESCRIPTOR – формиране на пикселен формат, включващ дълбочина.

HDC hDC;

void SetupPixelFormat(HDC hDC)

{

int nPixelFormat; // идентификатор на формата на пикселите

static PIXELFORMATDESCRIPTOR pfd = {

sizeof(PIXELFORMATDESCRIPTOR), // големина на
структурата

1, // версия

PFD_DRAW_TO_WINDOW | // поддръжка на рисуване
върху прозореца

PFD_SUPPORT_OPENGL | // поддръжка на OpenGL

PFD_DOUBLEBUFFER, // поддръжка на двоен буфер

PFD_TYPE_RGBA, // RGBA режима на цветовете

32, // дълбочина на цвета

0, 0, 0, 0, 0, 0, // празни

0, 0, // без alpha буфер

```

0,                // без accumulation буфер
0, 0, 0, 0,      // игнориране на accumulation
16,              // дълбочина на z-буфера
0,               // няма stencil буфер
0,               // няма auxiliary буфери
0,              // игнориране
0,               // резервиране на слоеве за рисуване
0, 0, 0 };      // игнориране на маските за рисуване

```

```

nPixelFormat = ChoosePixelFormat( hDC, &pfd ); //въвежда избрания
формат на пикселите

```

```

SetPixelFormat( hDC, nPixelFormat, &pfd ); // Установяване на
избрания формат на пикселите

```

```

}

```

WM_DESTROY – използва се за освобождаване на текущия рендиран контекст и връщане към основния прозорец.

```

hDC = GetDC(...);
wglMakeCurrent(hDC, NULL);
wglDeleteContext(...);

```

WM_SIZE – използва се за предефиниране на размерите на видимия прозорец, определяне на типа на проекцията, фоновия цвят на екрана, осветяването на обектите.

По подразбиране графиката се рендира на целия екран. Изчертаването може да бъде направено върху част от генерирания прозорец с помощта на

glViewport(GLint x, GLint y, GLsizei width, GLsizei height)

Първите два аргумента задават координатите на долния ляв ъгъл, а останалите два аргумента – широчината и височината на пространството, в което ще се осъществи рендирането.

Обслужването на проекцията, обектите и текстурите е реализирано чрез обща матрица на трансформациите с размерност 4 x 4. Към кой от изброените елементи ще се отнасят следващите трансформации, се определя с командата

glMatrixMode(mode);

Параметърът **mode** приема един от възможните аргументи

GL_MODELVIEW – прилага следващите матрични операции към стека на обект;

GL_PROJECTION - прилага следващите матрични операции към стека на проекцията;

GL_TEXTURE - прилага следващите матрични операции към стека на текстурата.

Всеки обслужван режим се въвежда в стека чрез **glLoadIdentity ()**.

Преди да се приложат трансформации за избрания режим, се зарежда начален вариант на матрицата, който елиминира предишното въведено състояние. Командата **glLoadIdentity ()** привежда матрицата на стека в начален вид и е еквивалентна на зареждане на състояние на матрицата чрез команда **glLoadMatrix** във вида:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Функцията **glGet(GL_аргумент)** връща:

текущия режим – за аргумент **GL_MATRIX_MODE**,
а чрез аргумент **GL_MODELVIEW_MATRIX**,
GL_PROJECTION_MATRIX или **GL_TEXTURE_MATRIX** – състоянието на съответната матрица.

Дефиниране на тип проекция

Типът проекция указва по какъв начин ще се наблюдават обектите в зависимост от разстоянието до наблюдателя. За целта са дефинирани 2 вида проекции:

Ортографска – обектът се вижда чрез успоредно проектиране върху равнината на наблюдение. Използва се функция

glOrtho(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top, GLdouble zNear, GLdouble zFar)

Функцията приема 6 аргумента. Първите четири определят координатите на лява, дясна, долна и горна изрязваща равнина от видимото за наблюдателя пространство, а последните два аргумента определят съответно координатите на най-близкия и най-далечния обект, който ще бъде показан. Функцията построява паралелепипед на перспективата на виждане със съответните координати. Всички обекти, които попадат в неговата вътрешност, са видими.

Перспективна – обектът се вижда чрез централно проектиране върху равнината на наблюдение. Използват се две функции, отличаващи се по начина на задаване на параметрите.

gluPerspective(GLdouble fovy, GLdouble aspect, GLdouble zNear, GLdouble zFar)

Функцията приема четири аргумента: първият определя зрителния ъгъл в у направление, който може да бъде от 0 - 180 (стандартно 45 градуса), вторият аргумент е аспект, той се генерира, като се раздели широчината на видимата площ на височината ѝ (x/y). За квадрат аспектът е равен на 1. Последните два аргумента са съответно най-близкото и най-далечното разстояние, на което ще бъдат видяни различните обекти.

glFrustum(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top, GLdouble znear, GLdouble zfar)

Функцията приема 6 аргумента. Първите четири аргумента определят площта на виждане при близко разстояние чрез задаване на координати на върховете, а последните два – съответно най-близкото и най-далечното разстояние.

И двете функции построяват пирамида, която обхваща видимото пространство.

Задаване на фонов цвят за екрана

Фоновият цвят се дефинира с:

glClearColor(GLclampf red, GLclampf green, GLclampf blue, GLclampf alpha)

Четири параметъра се задават със стойности между 0 и 1. Първите три определят цветообразуващите, а третият – прозрачността. Видимият цвят се определя от съотношението на трите цветообразуващи.

Избор на динамично осветление

За да бъдат видими обектите, е достатъчно да има подсветка на сцената, която отговаря на общо осветление и видимост на обектите. Всяко друго осветяване се задава с помощта на режима на отражение дифузно или огледално и вида на дефинирания светлинен източник.

Ambient – задава подсветката с минимална видимост и светлина идваща отвсякъде;

Diffuse – задава светлина, идваща от определена посока (източник), отразяваща се дифузно във всички посоки;

Specular – задава светлина, идваща от определена посока (източник) и отразяваща се огледално в определена посока.

Включване на динамичната светлина означава създаване на реалистично осветяване на обектите в зависимост от взаимното им положение в сцената. За целта към моделната матрица се прилагат:

- Включване на светлината в създадения прозорец
glEnable (GL_LIGHTING)
- Задаване на обща подсветка. Желателно е общата подсветка да се определя като глобална светлина. Това става с функцията :
glLightModelfv(GLenum pname, const GLfloat *params)

първи аргумент – константа GL_LIGHT_MODEL_AMBIENT,

втори аргумент – масив, определящ излъчването на Ambient светлината.

По подразбиране Ambient светлината в OpenGL е `ambient_light[] = { 0.2, 0.2, 0.2, 1.0 }`.

- Режим на отклик на материала

Използва функция на задаване в два варианта :

glMaterialf(GLenum face, GLenum pname, GLfloat param)

glMaterialfv(GLenum face, GLenum pname, const GLfloat *params)

Първият аргумент указва за коя част от обекта се отнася съответната дефиниция на материала, може да бъде една от константите GL_FRONT, GL_BACK или GL_FRONT_AND_BACK.

Вторият аргумент определя начина на отразяване на светлината – използва се с `glMaterialfv`

GL_AMBIENT – определя начина на отразяване на светлина тип Ambient.

GL_DIFFUSE – определя начина на отразяване на светлина тип Diffuse.

GL_SPECULAR – определя начина на отразяване на светлина тип Specular.

GL_AMBIENT_AND_DIFFUSE определя едновременно еднакъв начин на отразяване за Ambient и Diffuse светлината.

Третият аргумент на функцията е масив, който може да бъде същият, с който е дефинирано описанието на светлина от определен източник.

Излъчващ обект – използва се с `glMaterialfv`

GL_EMISSION – определя излъчвана от обекта светлина с данните от масива – трети аргумент на функцията. Тя се отразява единствено на цвета на обекта и се използва при създаване на самосветещ обект.

Големината на отражението при огледално отражение – използва се с `glMaterialf`

GL_SHININESS – определя големината на най-яркото отражение, причинено от светлина тип SPECULAR, чрез стойността на третия аргумент на функцията. Възможните стойности са между 0 и 128. Колкото по-голямо е то, толкова по-точно фокусирано и по-малко е светлинното петно.

Включването на светлинен източник води до промяна на цвета на обекта съобразно цвета на лъчението и начина на отразяване. Отчитане на цвета на обекта при формиране на крайния му цвят се задава чрез въвеждане на константа

`glEnable(GL_COLOR_MATERIAL)`

и определяне кои видове светлина да участват в крайния цвят на обекта. За целта се използва функцията:

`glColorMaterial(GLenum face, GLenum mode)` –

Първият аргумент може да бъде една от вече споменатите константи `GL_FRONT`, `GL_BACK` или `GL_FRONT_AND_BACK`, а вторият – `GL_EMISSION`, `GL_AMBIENT`, `GL_DIFFUSE`, `GL_SPECULAR` или `GL_AMBIENT_AND_DIFFUSE`.

- Тестване на разстоянията до обектите **`glEnable (GL_DEPTH_TEST)`**

Определянето на светлинен източник и позицията на наблюдателя е добре да бъдат направени в рендиращата процедура, изпълнявана при обработка на системно съобщение **WM_PAINT**.

- За дефиниране на светлинен източник се използва една от двете разновидности на функцията :

glLightf(GLenum light, GLenum pname, GLfloat param)

glLightfv(GLenum light, GLenum pname, const GLfloat *params)

Функцията има следните три аргумента:

Първият е име на светлинния източник, което се определя с константа-GL_LIGHT0, GL_LIGHT1, GL_LIGHT2, GL_LIGHT3, GL_LIGHT4, GL_LIGHT5, GL_LIGHT6 или GL_LIGHT7. Задавайки конкретно име на източника, чрез следващите във функцията параметри се описват неговите характеристики.

В OpenGL може да има едновременно осем действащи светлинни източника.

Вторият аргумент може да бъде:

Тип на светлината – използва се с glLightfv

GL_AMBIENT – определя подсветката.

GL_DIFFUSE – определя Diffuse излъчване на източника.

GL_SPECULAR – определя Specular излъчването на източника.

При избор на тип на светлината третият аргумент на функцията е масив, съдържащ параметрите на оцветяването във формат RGBA

Например: float m[] = { 0.2 , 0.2 , 0.2, 1.0 };

glLightfv(GL_LIGHT0 , GL_AMBIENT , m);

Позиция на източника– използва се с glLightfv

GL_POSITION – определя координатите на светлинния източник .

При този избор, третият аргумент на функцията е масив с хомогенните координати на източника (x , y , z , (w)). За w=0 източникът се намира в безкрайността, а x , y и z определят посоката на светлината.

Посока на светене за насочен източник – използва се с glLightfv

GL_SPOT_DIRECTION – определя посоката на разпространение на светлината.

При избор на посока третият аргумент на функцията е масив с координати на насочването на светлината.

Разпределение на яркостта в напречното сечение на снопа – използва се с glLightf

GL_SPOT_EXPONENT – определя фокуса на светлината.

При избор на разпределение на яркостта третият аргумент на функцията представлява число между 0.0 и 128.0, което показва максимума в центъра на снопа. Разпределението на яркостта в напречното сечение се променя по експоненциален закон, спадайки до 0.

Например `glLightf ( GL_LIGHT0 ,GL_SPOT_EXPONENT ,10.0 );`

Пространствен ъгъл на разпространение на светлината – използва се с `glLightf`

GL_SPOT_CUTOFF – определя ъгъла на конуса на разпространение.

При този избор третият аргумент на функцията е стойност на ъгъл в градуси от 0 до 90.

Например : `glLightf ( GL_LIGHT0 , GL_SPOT_CUTOFF , 25.0 );`

За един светлинен източник е необходимо да бъдат описани всички нужни параметри, преди той да бъде включен.

Включване на източник - **`glEnable( )`**. Например `glEnable(GL_LIGHT0)`.

Изключване на източник - **`glDisable( )`**.

- *Определяне на позиция на наблюдение на сцената*

Позицията на наблюдателя се задава с функцията

`gluLookAt ( eyex , eyey, eyez, centerx, centery, centerz, upx, upy, upz )`

Първите три аргумента определят местоположението на камерата (или преместването на цялата сцена около статичната точка на виждане), следващите 3 - координатите на гледната точка, към която е насочена камерата (или завъртването на сцената), а последните три определят вектора, който показва кое е "нагоре". Нормално се използва направлението на у оста или стойности за `upx`, `upy` и `upz` съответно 0 , 1 и 0.

- *Създаване на обект*

Поставяне на моделната матрица за обслужване в стека с фигурата

`glPushMatrix();`

Избор на метод на свързване на върховете на фигурата

Начинът, по който се свързват върховете в една фигура, се задава като аргумент при стартирането на самата фигура с функция **`glBegin (GLenum mode)`**

Вариантите са:

- `GL_POINTS` – остават отделни точки.
- `GL_LINES` – точките се свързват две по две в отделни линии. Ако броят на точките е нечетен, последната точка остава несвързана.
- `GL_LINE_STRIP` – всяка точка се свързва със следващата.
- `GL_LINE_LOOP` – всяка точка се свързва със следващата, като последната се свързва с първата.
- `GL_TRIANGLES` – точките се свързват в отделни триъгълници. Ако останат една или две точки, те не се свързват.
- `GL_TRIANGLE_STRIP` – точките се свързват в серия от триъгълници. Първо се свързват точки 1, 2 и 3, след това 3, 2 и 4, следват 3, 4 и 5, след това – 4, 5 и 6 и т.н.
- `GL_TRIANGLE_FAN` – точките се свързват отново в серия от триъгълници, но начинът на свързване е различен. Първо 1, 2 и 3 после 1, 3 и 4 след това 1, 4 и 5 и т.н. Така се получава подобие на диск, в което всички триъгълници имат една допирна точка.
- `GL_QUADS` – точките се свързват в отделни четириъгълници. Ако броят на точките не е кратен на 4, последните остават несвързани.
- `GL_QUAD_STRIP` – точките се свързват в серия от четириъгълници, като първо се свързват точки 1, 2, 4 и 3, следват 3, 4, 6 и 5, след това 5, 6, 8 и 7 и т.н.
- `GL_POLYGON` – точките се свързват в полигон, който трябва да е задължително изпъкнал, иначе са възможни аномалии.

Задаване на върхове и нормали

За създаване на прости обекти и тяхното осветяване е необходимо да се зададат координати за върховете и нормали към тях. Върховете се задават с функция

`glVertex{234}f[v]( x, y, z, (w) )`

Функцията е във варианти на брой аргументи: 2 – равнинна фигура; 3 – за **x, y, z координати** и 4 – за хомогенни координати. Координатите могат да бъдат зададени и в масив.

Нормала към връх се получава от векторното произведение между двойка вектори, имащи начало конкретния връх и край – съседни на него върхове. На практика това е посоката на нормалата към равнината, определена от тройката върхове. Тъй като към един връх принадлежат повече от една равнина, нормалата към връх е осреднена стойност от нормалите към прилежащите равнини. Въвеждането на пресметнатата нормала става с функцията:

glNormal3f(GLfloat nx, GLfloat ny, GLfloat nz) – приема 3 аргумента и това са x, y и z координатите на нормалния вектор.

Стойностите, които се въвеждат, са нормирани.

Задаване на цвят на фигура

Цветът на фигурата се определя в RGBA режим с функцията:

glColor{34}f[v](R, G, B, (A))

Възможните стойности са между 0 и 1. Липса на прозрачност се задава със стойност 1.0.

Цвят на фигурата в Color-Index режим се задава с функцията :

glIndex(index) – като число от 0 - 255 определя различен цвят, наличен в палитра.

При различни цветове на върховете функцията

glShadeModel(GLenum mode) определя начина на задаване на цвета.

mode може да приеме една от константите:

- **GL_SMOOTH** – за преливащ цвят, което е по подразбиране.
- **GL_FLAT** – за едноцветни фигури, независимо от цвета на върховете.

Край на фигурата

Описанието на всяка фигура, стартирано с **glBegin**, завършва с **glEnd ()**.

Ориентация на фигура

Веднъж създадена, всяка фигура подлежи на промяна на положението си в пространството. За целта се използва обща матрица 4x4 , резултат от всички приложени върху фигурата трансформации. За получаване на коефициентите е необходимо в началото да се активира **MODELVIEW** като аргумент на **glMatrixMode**.

glMatrixMode(GL_MODELVIEW);
glLoadIdentity();

Матрицата се въвежда в стека, а коефициентите ѝ зависят от приложените трансформации.

glPushMatrix();

следват възможни трансформации

glScale{fd}(x , y , z) – мащабира сцената. Подадените аргументи се използват като коефициент при мащабирането по x , y или z координата. Стойност по-малка от 1, води до смаляване на обекта, по-голяма от 1 – увеличаване на обекта. Ако някой аргумент е равен на 1, обектът запазва размерите си по съответната координата.

glRotate{fd}(angle , x , y , z) – завърта сцената/камерата като умножава x , y и z координатите по аргумента angle.

glTranslate{fd}(x , y , z) – премества сцената/камерата в зависимост от подадените координати.

Следва създаване на обект.

glPopMatrix();

Следва изваждане на матрицата от стека, обектът е ориентиран според нейните коефициенти.

Визуализация

Визуализацията на един буфер се прави с функцията **glFlush()**, а на два – с **SwapBuffers()** и изпращане на съобщение за прерисуване.